SOFTWARE ENGINEERING 2: OBJECT ORIENTED SOFTWARE ENGINEERING

1. This is a general question about C++ and Object Oriented Software Engineering.

   a) i) Describe in words the software architecture represented in the UML class diagram in Fig. 1.1.

[5]

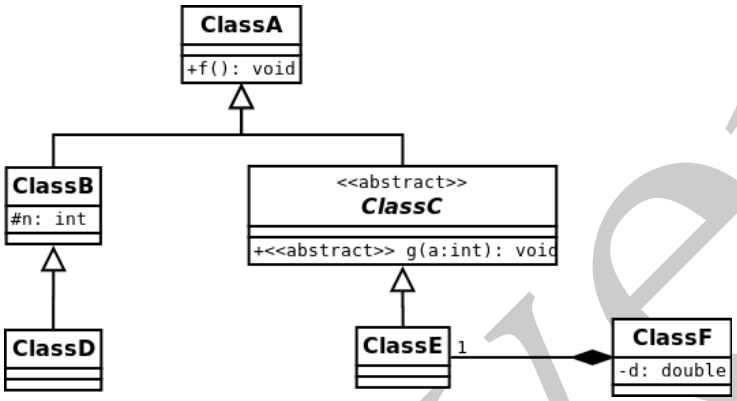


Figure 1.1 A UML class diagram

[bookwork, example analysis]

The diagram contain six classes. Classes `ClassB` and `ClassC` inherit from `ClassA`, `ClassD` inherits from `ClassB`, and `ClassE` inherits from `ClassC`.

Classes `ClassF` and `ClassE` are in a relationship of composition (each object instance of `ClassF` "*has*" one object instance of `ClassE`.

`ClassA` has a public member function `f` which has no parameters and doesn't return anything. `ClassC` is an abstract class. It has an abstract (i.e. pure virtual) member function `g` which has one `int` parameter and doesn't return anything. Since `ClassE` is not abstract, and inherits from the abstract class `ClassC`, then it necessarily provides an implementation for its member function `g`.

`ClassB` has a protected member data field n of type `int`. `ClassF` has a private member data field d of type `double`.

   ii) Write C++ declarations for all the classes in the UML class diagram in Fig. 1.1. The declarations can be kept to the essential skeleton (e.g. constructors can be omitted) but all the elements related to available information (including relationships) should be included.

[7]

Answers

[bookwork, example analysis, programming]

```
class ClassA {
    public:
        void f();
};


class ClassB : public ClassA {
    protected:
        int n;
};


class ClassC : public ClassA {
    public:
        virtual void g(int a) = 0;
};


class ClassD : public ClassB {

};


class ClassE : public ClassC {
    public:
        void g(int a);
};


class ClassF {
    private:
        double d;
        ClassE ce;
};
```

b) Explain the difference between the concepts of *Overloading*, *Redefining*, and *Overriding* member functions in C++. Illustrate your answer with an example in C++ code (including a `main()` function calling the overloaded, redefined, and overriden functions).

[12]

[bookwork, programming]

Overloading member functions is similar to overloading global functions: two or more functions can have the same name but different parameter lists (i.e. different signatures).

Following is an example of C++ member function `f` overloaded in a class `A`. It can be called with no, one, or two arguments.

Answers

```
#include <iostream>
class A {
    public:
        void f();
        void f(int n);
        void f(double d);
        void f(int n, int m);
};

int main() {
    A a;

    a.f();     // A::f()
    a.f(1);    // A::f(int)
    a.f(1.0);  // A::f(double)
    a.f(1, 2); // A::f(int, int)

    return 0;
}

void A::f() { std::cout << "A::f()" << std::endl; }
void A::f(int n) { std::cout << "A::f(int)" << std::endl; }
void A::f(double d) { std::cout << "A::f(double)" << std::endl; }
void A::f(int n, int m) { std::cout << "A::f(int, int)" << std::endl; }
```

Redefining and *Overriding* member functions in C++ need to be performed whithin a derived class inheriting from a base class, whereas *Overloading* can be performed independently from the concept of inheritance, as illustrated in the previous C++ code.

A derived class can change the definition of a member function inherited from a base class.

If the function is *virtual*, then the change is called *overriding*. If not, then it is called *redefining*.

Following is an example of two member functions `f1` and `f2` defined in a class `Base`, and respectively redefined and overriden in the `Derived` class.

```
#include <iostream>
class Base {
    public:
        void f1();
        virtual void f2();
};

class Derived : public Base {
    public:
        void f1(); // redefining (non-virtual function)
        void f2(); // overriding (virtual function)
};

int main() {
    Base* ptr = new Derived();
```

Answers

```
    ptr->f1(); // Base::f1()
    ptr->f2(); // Derived::f2()

    return 0;
}

void Base::f1() { std::cout << "Base::f1()" << std::endl; }
void Base::f2() { std::cout << "Base::f2()" << std::endl; }

void Derived::f1() { std::cout << "Derived::f1()" << std::endl; }
void Derived::f2() { std::cout << "Derived::f2()" << std::endl; }
```

Finally, as illustrated in the previous examples, overloading member functions leads to a *compile-time polymorphism*. And overriding virtual functions leads to a *run-time polymorphism*.

c) Consider the binary *addition* (+) operator.

Consider a class `A` having a private member variable `n` of type `int` and a constructor taking one `int` argument.

If `a1` and `a2` are two instances of class `A`, the addition `a1 + a2` can be defined as an object instance of class `A`, which member variable `n` equals the arithmetic addition of `a1.n` and `a2.n`.

i) Write C++ declaration and definition of class `A` overloading the addition operator as a *member function*.

[5]

[bookwork, programming]

```
class A {
    public:
        A(int n_in): n(n_in) {}

        A operator+(const A& other) {
            return A(n + other.n);
        }

    private:
        int n;
};
```

ii) Write C++ declaration and definition of class `A` overloading the addition operator as a *global function*.

[5]

[bookwork, programming]

```
class A {
```

Answers

```
    public:
        A(int n_in) : n(n_in) {}

        friend A operator+(const A& lhs, const A& rhs);

    private:
        int n;
};

A operator+(const A& lhs, const A& rhs) {
    return A(lhs.n + rhs.n);
}
```

iii) Consider the following snippet and the two compile-time errors:

```
int main() {
    A a1(0);

    A a2 = a1 + 1;
    a2 = 1 + a1;

    return 0;
}
```

```
$g++ prog.cpp -o prog
$[...] error: no match for operator+ (operand types are A and int)
     A a2 = a1 + 1;
                ^
$[...] error: no match for operator+ (operand types are int and A)
     a2 = 1 + a1;
              ^
```

Provide an appropriate overloading of the addition operator that handles such additions between `int` variables and objects of class `A`, and resolves the two previous compile errors.

(If `a` is an instance of class `A` and `num` is an `int`, the additions `a + num` and `num + a` can be defined as an object instance of class `A`, which member variable `n` equals the arithmetic addition of `a.n` and `num`.)

[6]

[bookwork, programming]

The addition operator+ is a binary operator, taking two operands: one left (lhs) and one right (rhs).

Overloading operator+ as a member function imposes the lhs operand to be of type `A`.

```
A operator+(int num) {
  return A(n + num);
}
```

Answers

```
int main() {
    A a1(0);

    A a2 = a1 + 1; // OK
    a2 = 1 + a1;   // not OK

    return 0;
}
```

Overloading operator+ as a global function is more suitable than overloading it as a member function because it gives more flexibility on the types of lhs and rhs.

```
A operator+(const A& lhs, int num);
A operator+(int num, const A& rhs);

int main() {
    A a1(0);

    A a2 = a1 + 1; // OK
    a2 = 1 + a1;   // OK

    return 0;
}
```

2. This question deals with memory management and container classes in C++.

Write the implementation (you can keep the definition and the declaration together) of a container class `intset` representing a *set* of integers, i.e. a data structure in which elements (in this case `int` numbers) are stored without any particular order, and without any repeated values.

Elements can be inserted in an `intset` (making the `intset` grow), and can be removed from it (making it shrink).

a) Declare suitable member data for the class `intset`. The implementation should be based on a dynamically allocated array.

[3]

b) Define a *constructor* which takes as argument an integer representing the initial physical size of the `intset`.

[3]

c) Define the *copy constructor* for the class `intset`.

[5]

d) Define the *assignment operator* for the class `intset`.

[5]

e) Define the *destructor* for the class `intset`.

Answers

[2]

f) Define a *contains* member function which returns true if an element (passed as argument) is in the `intset`, and false otherwise.

[2]

g) Define an *insert* member function which inserts an element (passed as argument) in the `intset`. If the element is already in the `intset`, this has no effect.

[4]

h) Overload the *stream insertion* (<<) operator for the class `intset`.

[4]

i) Define an *empty* member function (with no arguments) which returns true if the `intset` is empty and false otherwise.

[1]

j) Define a *size* member function (with no arguments) which returns the number of elements in the `intset`.

[1]

[bookwork, new example, programming]

```
#include <iostream>

class intset {
    public:

        // b)
        intset(int phsize_in) : phsize(phsize_in), lsize(0) {
            data = new int[phsize];
        }

        // c)
        intset(const intset& other) {
            lsize = other.lsize;
            phsize = other.phsize;
            data = new int[phsize];
            for(int i = 0; i < lsize; i++) {
                data[i] = other.data[i];
            }
        }

        // d)
        intset& operator=(const intset& other) {
            if(other.data != data) {
                delete[] data;
```

Answers

```
            lsize = other.lsize;
            phsize = other.phsize;
            data = new int[phsize];
            for(int i = 0; i < lsize; i++) {
                data[i] = other.data[i];
            }
        }
        return *this;
    }

    // e)
    ~intset() {
        delete[] data;
    }

    // f)
    bool contains(int n) {
        for(int i = 0; i < lsize; i++) {
            if(data[i] == n) {
                return true;
            }
        }
        return false;
    }

    // g)
    void insert(int n) {
        if(!contains(n)) {
            if(phsize == lsize) {
                phsize = phsize * 2;
                int* tmp = new int[phsize];
                for(int i = 0; i < lsize; i++) {
                    tmp[i] = data[i];
                }
                delete[] data;
                data = tmp;
            }
            data[lsize] = n;
            lsize = lsize + 1;
        }
    }

    // h)
    friend std::ostream& operator<<(std::ostream& out, const intset& s) {
        out << "{";
        for(int i = 0; i < s.lsize; i++) {
            out << s.data[i] << ", ";
        }
        out << "}";
        return out;
    }
```

Answers

```
        // i)
        bool empty() {
            return(lsize == 0);
        }

        // j)
        int size() {
            return lsize;
        }

    // a)
    private:
        int* data;
        int phsize; // physical size
        int lsize;  // logical size
};
```

3. This question deals with C++ templates and C++ exceptions.

   a) i) Write a template function which, given in input an `std::list` of elements of a generic type (passed by `const` reference), returns the *sum* of its elements. The function should throw a relevant exception if there is no way to return a meaningful sum. The implementation must use iterators.

[8]

[bookwork, programming]

```
#include <iostream>
#include <list>
#include <stdexcept>

template<typename T>
T sumlist(const std::list<T>& l) {
    if(l.begin() == l.end()) {
        std::logic_error err("no meaningful return for empty list");
        throw err;
    }

    typename std::list<T>::const_iterator it = l.begin();
    T sum = *it;
    it++;

    for(; it != l.end(); ++it) {
        sum = sum + *it;
    }

    return sum;
}
```

Answers

ii) Write a `main()` function containing examples of the function being called, including appropriate exception catching.

[6]

[bookwork, programming]

```
int main() {
    std::list<int> l1, l2;

    l1.push_back(1);
    l1.push_back(2);
    l1.push_back(3);

    std::list<std::string> l3;
    l3.push_back("I love ");
    l3.push_back("C++ ");
    l3.push_back("and OOP.");

    try {
        int sum1 = sumlist(l1);
        std::cout << "sum1 = " << sum1 << std::endl;
        // 6

        std::string sum3 = sumlist(l3);
        std::cout << "sum3 = " << sum3 << std::endl;
        // "I love C++ and OOP."

        int sum2 = sumlist(l2);
        std::cout << "sum2 = " << sum2 << std::endl;
        // exception
    }
    catch(std::logic_error e) {
        std::cout << "exception caught (empty list)" << std::endl;
    }

    return 0;
}
```

iii) Mention an example of a case in which the content of a list passed in input to the function would cause a compiler error, and explain why such error occurs.

[4]

[bookwork]

The template function `sumlist` is using the addition operator + (in `sum = sum + *it;`) and the assignment operator (in `T sum = *it;`). If the list contains a type for which one of these two operators is not overloaded, the result is a compiler error.

Answers

b) Consider the following code.

Trace its execution and its output (including an explanation for all the steps) using the three following definitions of function `f`.

```
#include <iostream>

class my_error {
    public:
        int num;
};

class A {
    public:
        A(int n) {
            if(n == 1) {
                my_error er;
                er.num = 999;
                throw er;
            }
        }
};

void f(); // see three different definitions in i) ii) and iii)

int main() {
    try {
        f();
    }
    catch(my_error er) {
        std::cout << "catch in main: " << er.num << std::endl;
    }
    std::cout << "end of main." << std::endl;

    return 0;
}
```

i) Function `f` is defined as:

```
void f() {
    try {
        A a(1);
    }
    catch(my_error er) {
        std::cout << "catch in f: " << er.num << std::endl;
    }

    std::cout << "end of f." << std::endl;
}
```

[4]

Answers

[bookwork, example analysis]

The program would generate the following output:

```
catch in f: 999
end of f.
end of main.
```

The function `f` is called in the main.

An exception is caught inside function `f`, caused by the constructor call `a(1)`. It is handled by the catch bloc of function `f` and 'catch in f: 999' is printed.

Then the following instructions of function `f` are executed, and 'end of f.' is printed.

The execution goes back to the main function, and 'end of main.' is printed.

ii) Function `f` is defined as:

```
void f() {
    try{
        A a(1);
    }
    catch(my_error er) {
        std::cout << "catch in f: " << er.num << std::endl;
        return;
    }

    std::cout << "end of f." << std::endl;
}
```

[4]

[bookwork, example analysis]

The program would generate the following output:

```
catch in f: 999
end of main.
```

As in i), the exception is caught by the try/catch bloc of function `f` and 'catch in f: 999' is printed.

But here, the execution of function `f` is stopped by the `return` instruction.

The execution goes back to the main function, and 'end of main.' is printed.

iii) Function `f` is defined as:

```
void f() {
    A a(1);
```

Answers

```
}
```

[4]

[bookwork, example analysis]

The program would generate the following output:

```
catch in main: 999
end of main.
```

The function `f` is called in the main.

The constructor call `a(1)` in function `f` is not in a try bloc. The exception caught is then handled by the try/catch bloc of the `main`, and 'catch in main: 999' is printed.

Then the following instructions of the `main` are executed, and 'end of main.' is printed.

Answers